

■ The Four Number Systems

System	Base	Digits Used	Prefix in C	Example
Decimal	10	0–9	none	255
Binary	2	0, 1	0b	0b11111111
Octal	8	0–7	0	0377
Hexadecimal	16	0–9, A–F	0x	0xFF

■ Decimal ↔ Binary Conversion

◆ Decimal to Binary (Division by 2 Method)

Divide the number repeatedly by 2, record remainders bottom-to-top:

Convert 45 to Binary:

```

45 ÷ 2 = 22 remainder 1 (LSB)
22 ÷ 2 = 11 remainder 0
11 ÷ 2 = 5 remainder 1
5 ÷ 2 = 2 remainder 1
2 ÷ 2 = 1 remainder 0
1 ÷ 2 = 0 remainder 1 (MSB)

```

Read from bottom to top: 45(10) = 101101(2)

◆ Binary to Decimal

Convert 101101(2) to Decimal:

```

1×2^5 + 0×2^4 + 1×2^3 + 1×2^2 + 0×2^1 + 1×2^0
= 32   + 0   + 8   + 4   + 0   + 1
= 45(10)

```

■ Hexadecimal

Decimal	Hex	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101

Decimal	Hex	Binary
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

■ 2's Complement (Signed Numbers)

Computers represent negative numbers using 2's complement: invert all bits, then add 1.

Find 2's complement of 45 (8-bit):

```
45 in binary   : 0010 1101
Invert bits    : 1101 0010  (1's complement)
Add 1          : 1101 0011  (2's complement = -45)
```

So -45 is stored as 1101 0011 in an 8-bit signed byte.

■ Using Number Literals in C

```
#include <stdio.h>
int main() {
    int dec = 255;        // Decimal
    int oct = 0377;       // Octal (leading 0)
    int hex = 0xFF;       // Hexadecimal (0x prefix)
    // int bin = 0b11111111; // Binary (C23 / GCC extension)

    printf("Dec: %d\n", dec);
    printf("Oct: %o\n", oct);
    printf("Hex: %x\n", hex);
    printf("Hex uppercase: %X\n", hex);
    return 0;
}
```